Basics

In [4]:
```
print("Here we go")
```

```
Here we go
```

In [7]:
```
A = [9, 3, 9, 3, 9, 7, 9]
print("Array A =", A)
```

```
Array A =  [9, 3, 9, 3, 9, 7, 9]
```

In [ ]:
```

```

In [29]:
```
name = "Ian"
type(name)
a_boolean = True
type(a_boolean)
```

Out[29]:
```
bool
```

In [30]:
```
if a_boolean:
    print(f"My name is {name} and I have an array called A with vlaues: {A}

a,b,c = 100, 200, 300
a

nums = [400,500,600]
type(nums)
a,b,c = nums
a
# note the block only returns (prints) the last value
```

```
My name is Ian and I have an array called A with vlaues: [9, 3, 9, 3, 9, 7,
9]
```

Out[30]:
```
100
```

In [33]:
```
a_dict = {"a": 100, "b": 200, "c": 300}
type(a_dict)
```

Out[33]:
```
dict
```

In [43]:
```
len(a_dict)
```

Out[43]:
```
3
```

In [47]:
```
a_global = 2000

def my_func():
    a_global = 1000
    print(f"Now a_global is {a_global}")
    return 1
```

```
print(f"The value of a global is {a_global}")
print(name)
```

```
The value of a global is 2000
Ian
```

In [59]:
```
a = 1000
f = float(a)
print(a, f, type(a), type(f))
```

```
1000 1000.0 <class 'int'> <class 'float'>
```

In [62]:
```
# random

import random

print(f"random [1-10]: {random.randint(1,10)}")
print(f"random float [0-1]: {random.random()}")
```

```
random [1-10]: 2
random float [0-1]: 0.6849451482265023
```

In [81]:
```
# lists
a = [5, 6, 7]


print(type(a))
print(a[0])
print(a[0:2])
print(a[-3])
# print(a[list("0","1")])
print(list([1,2,3]))
```

```
<class 'list'>
5
[5, 6]
5
[1, 2, 3]
```

In [78]:
```
# getting help - help, dir, __doc__
help(list.append)
```

```
Help on method_descriptor:

append(self, object, /) unbound builtins.list method
    Append object to the end of the list.
```

In [85]:
```
dir(list)
```

Out[85]:
```
['__call__',
 '__class__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
 '__name__',
 '__ne__',
 '__new__',
 '__qualname__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__self__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 '__text_signature__']
```

In [79]:
```
list.__doc__
```

Out[79]:
```
'Built-in mutable sequence.\n\nIf no argument is given, the constructor cre
ates a new empty list.\nThe argument must be an iterable if specified.'
```

In [101…
```
# strings
s = "Hello, World!"
s_multiline = """This is a multiline string
that spans multiple lines."""

print(f"String s: {s} \nString s_multiline: {s_multiline} \n{s[0:5]}")

for i in s:
    do_something = i
    # print(i)

for i in range(len(s)):
    print(s[i])

print(f"Length of string 'world': {len("world")}")
print(f"range(5): {list(range(5))}")
print(f"range(5): {range(5)}")

text = "  Now   is  the     time  -  of our discontent. "
```

```python
# upper, lower, replace, strip, split, join
print(f"Original text: '{text}'")
print(f"Uppercase: '{text.upper()}'")
print(f"Lowercase: '{text.lower()}'")
print(f"Stripped: '{text.strip()}'")
print(f"Replaced: '{text.replace(" ", "_")}'")
print(f"Split: {text.split()}")
help(str.split)
print(f"Split: {text.split("-")}")
print(f"Split: {text.split()}")
print(f"Join: {'-'.join(text.split())}")

# concat
split_text = text.split()
new_text = split_text[0] + " " + split_text[3]
print(f"New text: '{new_text}'")

text = "hello world"
# count, startswith, endswith, find, isalnum, isalpha
print(f"Count 'o': {text.count('o')}")
print(f"Starts with 'hello': {text.startswith('hello')}")
print(f"Ends with 'world': {text.endswith('world')}")
print(f"Find 'world': {text.find('world')}")
print(f"Is alphanumeric: {text.isalnum()}")
print(f"Is alphabetic: {'hello'.isalpha()}")
```

```
String s: Hello, World!
String s_multiline: This is a multiline string
that spans multiple lines.
Hello
H
e
l
l
o
,

W
o
r
l
d
!
Length of string 'world': 5
range(5): [0, 1, 2, 3, 4]
range(5): range(0, 5)
Original text: '  Now   is  the     time  -  of our discontent. '
Uppercase: '  NOW   IS  THE     TIME  -  OF OUR DISCONTENT. '
Lowercase: '  now   is  the     time  -  of our discontent. '
Stripped: 'Now   is  the     time  -  of our discontent.'
Replaced: '__Now___is__the_____time__-__of_our_discontent._'
Split: ['Now', 'is', 'the', 'time', '-', 'of', 'our', 'discontent.']
Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1) unbound builtins.str method
    Return a list of the substrings in the string, using sep as the separato
r string.

      sep
        The separator used to split the string.

        When set to None (the default value), will split on any whitespace
        character (including \n \r \t \f and spaces) and will discard
        empty strings from the result.
      maxsplit
        Maximum number of splits.
        -1 (the default value) means no limit.

    Splitting starts at the front of the string and works to the end.

    Note, str.split() is mainly useful for data that has been intentionally
    delimited.  With natural text that includes punctuation, consider using
    the regular expression module.

Split: ['  Now   is  the     time  ', '  of our discontent. ']
Split: ['Now', 'is', 'the', 'time', '-', 'of', 'our', 'discontent.']
Join: Now-is-the-time---of-our-discontent.
New text: 'Now time'
Count 'o': 2
Starts with 'hello': True
Ends with 'world': True
Find 'world': 6
```

```
Is alphanumeric: False
Is alphabetic: True
```

In [104…
```python
# boolean logic
print(f"5 == 10: {5 == 10}")
print(f"5 != 10: {5 != 10}")
print(f"5 > 10: {5 > 10}")
print(f"cmp int to list: {5 != [1,2,3,4,5]}")
```

```
5 == 10: False
5 != 10: True
5 > 10: False
5 < 10: True
```

In [113…
```python
# operators: +, -, *, /, //, %, **, +=, -=, *=, /=, //=, %=, **=
a = 10
b = 3
print(a % b)
print(a // b)
print(a ** b)
a+=3
print(a)
```

```
1
3
1000
13
```

In [ ]:
```python

```

In [118…
```python
# logicals

a = True
b = False
a and b
```

Out[118… False

In [121…
```python
# identity operators
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(f"a is c: {a is c}")
print(f"a is b: {a is b}")
print(f"a == b: {a == b}")
print(f"a is not b: {a is not b}")
```

```
a is c: True
a is b: False
a == b: True
a is not b: True
```

In [166…
```python
fruits = ["apple", "banana", "cherry"]
text = "Hello World"

print(f"'banana' in fruits: {'banana' in fruits}")
```

```
print(f"Slice [1:3]: {fruits[1:2]}") # note the slice is exclusive of the er

# help(list)
# change, insert, remove, pop, sort, reverse, delete, clear
fruits[1] = "plaintain"
print(f"Change banana for plaintain {fruits}")
fruits.insert(1, "mango")
print(f"Insert mango {fruits}")
x = "cherry"
fruits.remove(x)
print(f"Remove cherry: {fruits}")
x = fruits.pop()
print(f"Pop fruits is: {x}, leaving fruitsd as: {fruits}.")
fruits.sort(reverse = True)
print(f"Sort fruits is: {fruits}.")
fruits.reverse()
print(f"Reverse fruits is: {fruits}.")
fruits.clear()
print(f"Cleared fruits is: {fruits}.")
```

```
'banana' in fruits: True
Slice [1:3]: ['banana']
Change banana for plaintain ['apple', 'plaintain', 'cherry']
Insert mango ['apple', 'mango', 'plaintain', 'cherry']
Remove cherry: ['apple', 'mango', 'plaintain']
Pop fruits is: plaintain, leaving fruitsd as: ['apple', 'mango'].
Sort fruits is: ['mango', 'apple'].
Reverse fruits is: ['apple', 'mango'].
Cleared fruits is: [].
```

In [147…

```
# in place operations vs new lists
numbers = [5, 2, 8, 1, 9]
print(f"Numbers original: {numbers}")
new_numbers = sorted(numbers)
print(f"New numbers is a sorted copy: {new_numbers}")
print(f"But numbers are still: {numbers}")
numbers.sort()
print(f"Numbers sorted in place {numbers}")
```

```
Numbers original: [5, 2, 8, 1, 9]
New numbers is a sorted copy: [1, 2, 5, 8, 9]
But numbers are still: [5, 2, 8, 1, 9]
Numbers sorted in place [1, 2, 5, 8, 9]
```

In [ ]:

```
# fibonacci series
a, b = 0, 1 # must be declared in python 3

while a < 10:
    print(a)
    a, b = b, a+b
```

```
0
1
1
2
3
5
8
```

In [ ]:
```python
def is_anagram(s1, s2):
    return set(s1) == set(s2)

def is_reverse(s1,s2):
    list1 = list(s1)
    # list2 = list(s2).reverse() <== does not work - must be two steps
    list2 = list(s2)
    list2.reverse() # must include () - otherwise does not work
    return list1 == list2


print(is_anagram("elvis", "lives"))

print(is_reverse("evil", "live"))

x = list("evil")
print(x)
y = list("live")
print(y)
z = list("accordian")
print(z)
z.reverse()
print(z)
```

```
True
False
['e', 'v', 'i', 'l']
['l', 'i', 'v', 'e']
['a', 'c', 'c', 'o', 'r', 'd', 'i', 'a', 'n']
['n', 'a', 'i', 'd', 'r', 'o', 'c', 'c', 'a']
```

In [4]:
```python
import numpy as np
import pandas as pd
```

In [2]:
```python
import plotly.express as px
fig = px.scatter(x=[0, 1, 2, 3, 4], y=[0, 1, 4, 9, 16])
fig.show()
```

In [5]:
```python
dates = pd.date_range("20130101", periods=6)
dates
```

Out[5]:
```
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[us]', freq='D')
```

In [12]:
```python
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=["A", "B", "C"
df
```

Out[12]:

	A	B	C	D
2013-01-01	-2.654466	1.211980	0.316425	1.496294
2013-01-02	1.341773	1.439307	0.663686	0.844195
2013-01-03	-0.934081	-1.592782	1.127901	-0.108961
2013-01-04	-0.159891	1.114518	-0.203416	0.542702
2013-01-05	-0.136289	-0.585143	-0.049495	-0.376867
2013-01-06	-0.022880	0.859218	-1.227854	-1.571055

In [15]:
```
df2 = pd.DataFrame(
    {
        "A": 1.0,
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)), dtype="float32"),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test", "train"]),
        "F": "foo",
    }
)
df2
```

Out[15]:

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

In [16]:
```
df2.dtypes
```

Out[16]:
```
A           float64
B    datetime64[us]
C           float32
D             int32
E          category
F               str
dtype: object
```

In [18]:
```
df.head(1)
```

Out[18]:

	A	B	C	D
2013-01-01	-2.654466	1.21198	0.316425	1.496294

In [19]:
```
df.tail(2)
```

Out[19]:

	A	B	C	D
2013-01-05	-0.136289	-0.585143	-0.049495	-0.376867
2013-01-06	-0.022880	0.859218	-1.227854	-1.571055

In [20]:
```
df.columns
```

Out[20]:
```
Index(['A', 'B', 'C', 'D'], dtype='str')
```

In [23]:
```
df.describe()
```

Out[23]:

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	-0.427639	0.407850	0.104541	0.137718
std	1.316546	1.216734	0.812237	1.072791
min	-2.654466	-1.592782	-1.227854	-1.571055
25%	-0.740533	-0.224052	-0.164936	-0.309891
50%	-0.148090	0.986868	0.133465	0.216870
75%	-0.051232	1.187615	0.576871	0.768821
max	1.341773	1.439307	1.127901	1.496294

In [24]:
```
df.T
```

Out[24]:

	2013-01-01	2013-01-02	2013-01-03	2013-01-04	2013-01-05	2013-01-06
A	-2.654466	1.341773	-0.934081	-0.159891	-0.136289	-0.022880
B	1.211980	1.439307	-1.592782	1.114518	-0.585143	0.859218
C	0.316425	0.663686	1.127901	-0.203416	-0.049495	-1.227854
D	1.496294	0.844195	-0.108961	0.542702	-0.376867	-1.571055

In [26]:
```
df
```

Out[26]:

	A	B	C	D
2013-01-01	-2.654466	1.211980	0.316425	1.496294
2013-01-02	1.341773	1.439307	0.663686	0.844195
2013-01-03	-0.934081	-1.592782	1.127901	-0.108961
2013-01-04	-0.159891	1.114518	-0.203416	0.542702
2013-01-05	-0.136289	-0.585143	-0.049495	-0.376867
2013-01-06	-0.022880	0.859218	-1.227854	-1.571055

In [27]:
```
df.sort_values(by="B")
```

Out[27]:

	A	B	C	D
2013-01-03	-0.934081	-1.592782	1.127901	-0.108961
2013-01-05	-0.136289	-0.585143	-0.049495	-0.376867
2013-01-06	-0.022880	0.859218	-1.227854	-1.571055
2013-01-04	-0.159891	1.114518	-0.203416	0.542702
2013-01-01	-2.654466	1.211980	0.316425	1.496294
2013-01-02	1.341773	1.439307	0.663686	0.844195

In [28]:
```
df.loc
```

Out[28]:
```
<pandas.core.indexing._LocIndexer at 0x1142a60d0>
```

In [29]:
```
dates[0]
```

Out[29]:
```
Timestamp('2013-01-01 00:00:00')
```

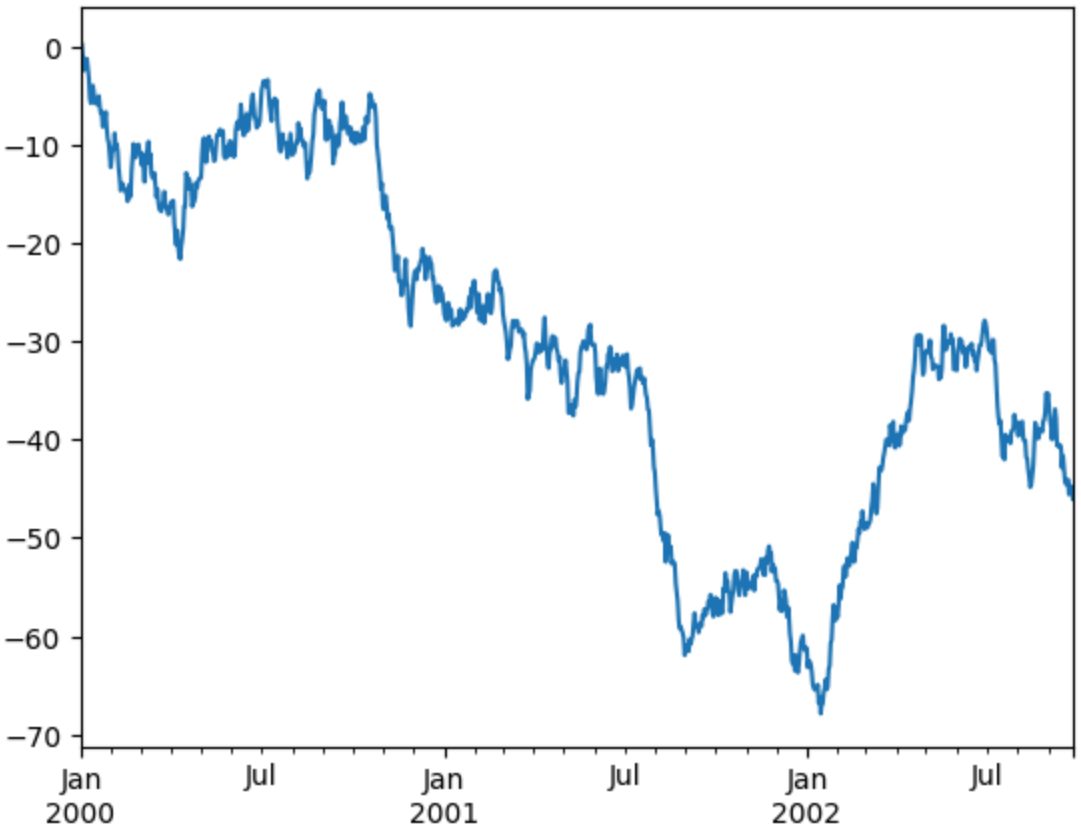
In [22]:
```
np.random.seed(123456)

ts = pd.Series(np.random.randn(1000), index=pd.date_range("1/1/2000", perioc

ts = ts.cumsum()

ts.plot();
```

```
Matplotlib is building the font cache; this may take a moment.
```



In [32]:
```
df["A"]
```

Out[32]:
```
2013-01-01   -2.654466
2013-01-02    1.341773
2013-01-03   -0.934081
2013-01-04   -0.159891
2013-01-05   -0.136289
2013-01-06   -0.022880
Freq: D, Name: A, dtype: float64
```

In [33]:
```
df["A"].value_counts()
```

Out[33]:
```
A
-2.654466    1
 1.341773    1
-0.934081    1
-0.159891    1
-0.136289    1
-0.022880    1
Name: count, dtype: int64
```

In [ ]:
```
# check if a number is in a list
#
l = [1,2,3,4]
3 in l
print(f"is 3 in the list: {3 in l}")
```

```
is 3 in the list True
```

In [ ]:
```python
# find a duplicate number in an integer list
l = [1,2,3,4,4]

unique_l = set(l)
if len(l) == len(unique_l):
    print("all values are unique")
elif len(l) > len(unique_l):
    print("there are duplicate values")

# keep a dict of counts of occurrences
seen = {k:0 for k in set(l)}

for n in l:
    # if n in seen.keys():
        seen[n] += 1

for key in seen.keys():
    if seen[key] > 1:
        print(f"{item} was seen {seen[item]} times.")
```

```
there are duplicate values
4 was seen 2 times.
```

In [ ]:
```python
# find duplicates in a list

l = [1,2,3,4,5,4]
for i in set(l):
    if l.count(i) > 1:
        print(f"value {i} occurs {l.count(i)} times")
```

```
value 4 occurs 2 times
```

In [41]:
```python
# find duplicates in a list and first positions at which they occur

l = [1,2,3,4,5,4]
duplicates = {}
for i in set(l):
    if l.count(i) > 1: # inefficient because l is scanned each time O(N2)
        print(f"value {i} occurs {l.count(i)} times")
        duplicates[i] = l.count(i)

duplicates

for k in duplicates.keys():
    pos = l.index(k)
    print(f"item {k} is duplicated at occurs first at position {pos}")
```

```
value 4 occurs 2 times
item 4 is duplicated at occurs first at position 3
```

In [ ]:
```python
# initiate a dictionary using a comprehension - set to zeros for each key

keys = range(1,10,2) # some set of keys (or use set(list) to make a unique s
dd = {x:0 for x in keys} # this is a dictionary comprehension - not a anon f
dd
```

Out[ ]:
```
{1: 0, 3: 0, 5: 0, 7: 0, 9: 0}
```

In [32]:
```
# find duplicates in a list of numbers

def find_duplicates(elements):
    duplicates, seen = set(), set()
    for element in elements:
        if element in seen:
            duplicates.add(element)
        seen.add(element)
    return list(duplicates)

find_duplicates([1,2,3,4,4,4])
```

Out[32]:
```
[4]
```

In [33]:
```
# check if two strings are anagrams

s1 = "elvis"
s2 = "lives"

set(s1) == set(s2)
```

Out[33]:
```
True
```

In [46]:
```
# remove all duplicates from list

l = [1,2,3,4,4]
l=list(set(l))
l
```

Out[46]:
```
[1, 2, 3, 4]
```

In [54]:
```
# find pairs of integers in list so that their sum is equal to integer x

def find_pairs(l,x):
    pairs = []
    for a in l:
        for b in l:
            # print(f"a + b is {a} + {b} = {a+b}")
            if a + b == x:
                pairs.append([a,b])
    return(pairs)


l = [1,2,3,4,5]
pairs = find_pairs(l, 9)

pairs
```

Out[54]:
```
[[4, 5], [5, 4]]
```

In [64]:
```
# enumerate - for iterating over upper half of matrix made by permuting a
```

```
l = [1,2,3,4,5]
for i, el_1 in enumerate(l):
    for j, el_2 in enumerate(l[i+1:]):
        # print(f"i: {i}, j: {j}")
        print(f"el_1: {el_1} el_2: {el_2}")


print(l[0+1:])
print(l[:])
print(l[:len(l)])
```

```
el_1: 1 el_2: 2
el_1: 1 el_2: 3
el_1: 1 el_2: 4
el_1: 1 el_2: 5
el_1: 2 el_2: 3
el_1: 2 el_2: 4
el_1: 2 el_2: 5
el_1: 3 el_2: 4
el_1: 3 el_2: 5
el_1: 4 el_2: 5
[2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

In [77]:
```
# filter

l = [1,2,3,4,5]
filter(lambda x: x>2, l)
list(filter(lambda x: x>2, l))
```

Out[77]: [3, 4, 5]

In [74]:
```
# map

# with one iterable
map(lambda x: x[0], ["apple", "banana", "cranberry"])

this_list = list(map(lambda x: x[0], ["apple", "banana", "cranberry"]))
print(this_list)

# with multiple iterables
another_list = list(map(lambda x,y: str(x) + " " + y + "s", [0,2,2], ["apple
print(another_list)
```

```
['a', 'b', 'c']
['0 apples', '2 bananas', '2 cranberrys']
```

In [ ]:
```
# reduce

l = list(map(lambda x: x**2, [1,2,3,4,5]))
```

In [ ]:
```
# check if a string is a palindrome

this_string = "backcab"
print(type(this_string))
```

```
def is_palindrome(s):
    r = list(s)
    r.reverse() # or use reverse(r)
    if list(s) == r:
        return True
    else:
        return False

# simpler
def is_palindrome(s):
    return s == s[::-1]

print(is_palindrome(this_string))
```

```
<class 'str'>
True
```

In [118…

```
# reverse
# methods like .reverse, .append, .sort act on lists in place and return 'No

fruits = ['apple', 'banana', 'cherry']
print(fruits)

# prints the operator
# fruits.reverse() alters the list in place and returns None
# print(fruits.reverse()) - just prints None

# instead
fruits.reverse()
print(fruits)

# there are corresponding iterator functions that return iterators
r = reversed(fruits)
# prints iterator
print(f"reversed(fruits) returns an iterator: {r}")
# instead
print(f"but this can be casted as a list: {list(r)}")

# sorted
s = sorted(fruits)
print(f"but sorted(fruits) returns a sorted list fruits: {s}")

# slicing can also be used to reverse and return copies
print(f"fruits is now: {fruits}")
rf = fruits[::-1] # read as start, stop, step
print(f"slicing can be used to return a reversed list {rf}")
fs = "fruits"
rfs = fs[::-1]
print(f"slicing casts strings as lists so {fs} can be turned to {rfs}")
```

```
['apple', 'banana', 'cherry']
['cherry', 'banana', 'apple']
reversed(fruits) returns an iterator: <list_reverseiterator object at 0x1094
5e1d0>
but this can be casted as a list: ['apple', 'banana', 'cherry']
but sorted(fruits) returns a sorted list fruits: ['apple', 'banana', 'cherr
y']
fruits is now: ['cherry', 'banana', 'apple']
slicing can be used to return a reversed list ['apple', 'banana', 'cherry']
slicing casts strings as lists so fruits can be turned to stiurf
```

In [ ]:
```python
# recursion

def exponential(n):
    # base case
    if n == 1:
        return 1

    # recursive case
    return n * exponential(n-1)

answer = exponential(5)

answer
```

Out[ ]: 120

In [ ]:
```python
# stack

# Push the values 1, 2, and 3 onto
# the stack, then pop one value off and print both the popped value and the

# LIFO
# use for keeping browser history queue for the back button

stack = []

stack.append(1)
stack.append(2)
stack.append(3)
print(f"stack starts as: {stack}")

p = stack.pop()

print(f"popped: {p}")
print(f"stack is now: {stack}")
```

```
stack starts as: [1, 2, 3]
popped: 3
stack is now: [1, 2]
```

In [134…
```python
# queue

# FIFO
# use for printer jobs and being polite
```

```
queue = []

# Add items to the queue
queue.append("Alice")
queue.append("Bob")
queue.append("Charlie")

print("Queue after additions:", queue)

# Remove the first item
removed_item = queue.pop(0)

print("Removed item:", removed_item)
print("Remaining queue:", queue)

# can also be implemented in reverse using insert(0, item) and pop()
# both methods are inefficent pop(0) and insert(0,) are O(n) time

# instead use dequeue

from collections import deque

queue = deque()

queue.append("Alice")
queue.append("Bob")
queue.append("Charlie")

print(f"using dequeue, pop(left) will be: {queue.popleft()}")
```

```
Queue after additions: ['Alice', 'Bob', 'Charlie']
Removed item: Alice
Remaining queue: ['Bob', 'Charlie']
using dequeue, pop(left) will be: Alice
```

In [146…

```
# get missing numbers in 1 - 100

a = list(range(1,101)) # a contains all number 1-100
r = list(range(1,101))
r.remove(5)            # remove one item
remove_items = [1,4,6] # or remove multiple items
r = [x for x in r if x not in remove_items]

def get_missing(a, r):
    missing = list(filter(lambda x: x not in r, a))
    return(missing)

def get_missing(a, r):
    a = set(a)
    r = set(r)
    return a - r

missing = get_missing(a,r)
print(f"missing nums are: {missing}")
```

```
missing nums are: {1, 4, 5, 6}
```

In [147…
```python
# intersection of two lists

def get_intersection(a,b):
    return list(set(a) & set(b))

a = [1,2,3,4,5]
b = [4,5,6,7,8]
answer = get_intersection(a,b)

print(f"the intersection is: {answer}")
```

```
the intersection is: [4, 5]
```

In [150…
```python
# find max and min in an unsorted list

print(max([1,2,3,4,5]))
print(min([2,1,4,2,6]))
```

```
5
1
```

In [ ]:
```python
#
```